

XCSdk Scanner SDK User Guide

SDK initialize

After init SDK, you can use scan function provided by scan service via APIs. Currently, there are two ways to initialize SDK:

```
XcBarcodeScanner.init(Context context, ScannerResult scannerResult)

XcBarcodeScanner.init(Context context, ScannersSymResult scannersSymResult)
```

Callback interface:

```
public interface ScannerResult {
    void onResult(String result);
}

public interface ScannersSymResult {
    void onResult(String sym, String barCode);
}
```

After init SDK, your application will connect with scan service, and the scan result will be output via the ScannerResult/ScannerSymResult callback.

Sample code:

```
XcBarcodeScanner.init(this, new ScannerResult() {
    @Override
    public void onResult(String result) {
        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                allResult = allResult + "\n" + result;
                mTextResult.setText(allResult);
                scrollToBottom();
            }
        });
    }
});

XcBarcodeScanner.init(this, new ScannersSymResult() {
    @Override
    public void onResult(String sym, String barCode) {
        runOnUiThread(new Runnable() {
            @Override
            public void run() {
                allResult = allResult + "\n" + sym + ":" + barCode;
                mTextResult.setText(allResult);
                scrollToBottom();
            }
        });
    }
});
```

SDK deinitialize

When the activity been destroyed or switched to background, we need de-initialize the SDK.

Sample code:

```
@Override
protected void onPause() {
    super.onPause();

    XcBarcodeScanner.deInit(this);
}
```

Query license status

Use the following API to query license status.

```
XcBarcodeScanner.getLicenseState();
```

All license status defined in the class *LicenseState*:

```
public class LicenseState {
    public static final int INACTIVE = 0;    // License not activated.
    public static final int ACTIVATING = 1;  // License is activating.
    public static final int ACTIVED = 2;     // License already activated.
    public static final int INVALID = 3;     // License invalid, contact
    vendor.
    public static final int NETWORK_ISSUE = 4; // Network issue, connet
    before active license.
    public static final int EXPIRED = 5;     // License expired, contact
    vendor.
}
```

Active license

Use the following API to active license if needed.

```
XcBarcodeScanner.activateLicense();
```

After active license, it need about 1 or 2 minutes to process. We can use the API *getLicenseState* to query license status.

Note: Active license need network connection.

Reactivate license

Use the following API to reactivate license if needed.

```
XcBarcodeScanner.forceNetworkActivate();
```

After active license, it need about 1 or 2 minutes to process. We can use the API *getLicenseState* to query license status.

Note: Active license need network connection.

Start/Stop scanning

Use the following API to control scan service to start or stop scanning.

```
XcBarcodeScanner.startScan();  
XcBarcodeScanner.stopScan();
```

Suspend/Resume scan service

Use the following API to suspend or resume scan service.

After suspend, any API to control scan start or stop will be bypassed, and camera resource will be released by scan service.

```
XcBarcodeScanner.suspendScanService();  
XcBarcodeScanner.resumeScanService();
```

Get scan service status

Use the following API to check current scan service status.

```
boolean isScanServiceSuspending();
```

Get version of scan service

Refer to the following sample code to get version info of scan service.

```
String serviceVer = XcBarcodeScanner.getServiceVersion();  
Log.d(TAG, "Service ver: " + serviceVer);
```

Get version of SDK

Refer to the following sample code to get current scanner SDK version.

```
String sdkVer = XcBarcodeScanner.getSdkVersion();  
Log.d(TAG, "SDK ver: " + sdkVer);
```

Advanced function

Enable/disable specific barcode support

Use the following API to enable or disable specific type of barcode support.

```
XcBarcodeScanner.enableBarcodeType(String barcodeType, boolean enabled);
```

All barcode types defined in the class BarcodeType.

```
public class BarcodeType {
    public static final String AZTEC = "Aztec";
    public static final String CODE11 = "Code11";
    public static final String CODE39 = "Code39";
    public static final String CODE93 = "Code93";
    public static final String CODE128 = "Code128";
    public static final String CODABAR = "Codabar";
    public static final String DOTCODE = "Dotcode";
    public static final String CODABLOCKF = "CODABLOCK F";
    public static final String DATAMATRIX = "DATAMATRIX";
    public static final String GS1_DATAMATRIX = "GS1DATAMATRIX";
    public static final String EAN8 = "EAN-8";
    public static final String EAN13 = "EAN-13";
    public static final String GS1_DATABAR = "GS1_DATABAR";
    public static final String HANXIN = "HANXIN";
    public static final String HK25 = "HK25";
    public static final String ITF25 = "ITF25";
    public static final String MATRIX25 = "MATRIX25";
    public static final String MAXICODE = "MAXICODE";
    public static final String MSI = "MSI";
    public static final String MICROPDF = "MICROPDF";
    public static final String PDF417 = "PDF417";
    public static final String USPS4ST = "USPS4ST";
    public static final String QRCODE = "QRCODE";
    public static final String INDUSTRIAL25 = "INDUSTRIAL25";
    public static final String TELEPEN = "TELEPEN";
    public static final String UPCA = "UPC-A";
    public static final String UPCE = "UPC-E";
    public static final String GS1_128 = "GS1-128";
    public static final String GS1_DATABAR_LIMITED = "GS1_DataBarLimited";
    public static final String GS1_DATABAR_EXPANDED = "GS1_DataBarExpanded";
    public static final String IATA25 = "IATA25";
    public static final String GRIDMATRIX = "Grid_Matrix";
}
```

Sample code:

```
XcBarcodeScanner.enableBarcodeType(BarcodeType.QRCODE, true); // Enable
QRCode support.
XcBarcodeScanner.enableBarcodeType(BarcodeType.QRCODE, false); // Disable
QRCode support.
```

Check specific type of barcode support status

All barcode types defined in the class BarcodeType.

```
boolean isBarcodeTypeEnabled(String type)
```

Config output method

Use the following API to config scan result output method.

```
XcBarcodeScanner.setOutputMethod(String method);
```

All output method defined in the class OutputMethod.

```
public class OutputMethod {  
    public static final String NONE = "NONE"; // No putput, application can  
    get barcode data via callback.  
    public static final String BROADCAST = "BROADCAST_EVENT"; // Output  
    barcode data via broadcast.  
    public static final String KEYBOARD = "KEYBOARD_EVENT"; // Output barcode  
    data via keyboard simulate.  
    public static final String CLIPBOARD = "CLIPBOARD_EVENT"; // Output  
    barcode data via clipboard simulate.  
    public static final String BROADCAST_KEYBOARD =  
    "BROADCAST_EVENT/KEYBOARD_EVENT"; // Output barcode data via broadcast and  
    keyboard simulate.  
    public static final String BROADCAST_CLIPBOARD =  
    "BROADCAST_EVENT/CLIPBOARD_EVENT"; // Output barcode data via broadcast and  
    clipboard simulate.  
}
```

Sample code:

```
XcBarcodeScanner.setOutputMethod(OutputMethod.CLIPBOARD);
```

Config barcode output event notification

Use the following API to config type of notification when scan success.

```
XcBarcodeScanner.setSuccessNotification(String notification);
```

All types of notifications defined in the class NotificationType.

```
public class NotificationType {  
    public static final String MUTE = "Mute";  
    public static final String SOUND = "Sound";  
    public static final String VIBRATOR = "Vib";  
    public static final String SOUND_VIBRATOR = "Sound/Vib";  
}
```

Sample code:

```
XcBarcodeScanner.setSuccessNotification(NotificationType.SOUND);
```

Config barcode scan LED indicator

Use the following API to enable or disable scan LED indicator.

```
XcBarcodeScanner.enableSuccessIndicator(boolean enabled);
```

Sample code:

```
XcBarcodeScanner.enableSuccessIndicator(true); // Enable LED indicator.  
XcBarcodeScanner.enableSuccessIndicator(false); // Disable LED indicator.
```

Config scan time limit

Use the following API to config scan time limitation.

```
XcBarcodeScanner.setTimeout(int seconds);
```

The range of time limitation: 1 ~ 9 seconds.

Sample code:

```
XcBarcodeScanner.setTimeout(5); // Set scan time limit to 5 seconds.
```

Config aimer light

Use the following API to config aimer light work mode.

```
void setAimerLightsMode(int aimMode);
```

Aimer light work mode defined in the class *AimerMode* :

```
public class AimerMode {  
    public static int ALWAYS_OFF; // Always off.  
    public static int TRIGGER_ON; // TurnOn while scanning.  
    public static int ALWAYS_ON; // Always On.  
}
```

Sample code:

```
XcBarcodeScanner.setAimerLightsMode(AimerMode.TRIGGER_ON); // Set aimer mode  
to: Turn on while scanning.  
XcBarcodeScanner.setAimerLightsMode(AimerMode.ALWAYS_OFF); // Set aimer mode  
to: always off.
```

Note: For laser aimer, DO NOT USE THE *ALWAYS_ON* mode.

Config illume light

Use the following API to config illume light work mode.

```
void setFlashLightsMode(int flashMode);
```

The work mode of illume light defined in class *FlashMode* :

```
public class FlashMode {  
    public static int OFF; // Always off.  
    public static int ILLUME_ONLY; // Only illume.  
    public static int ILLUME_STROBE; // Illume and Strobe.  
}
```

Sample code:

```
XcBarcodeScanner.setFlashLightsMode(FlashMode.ILLUME_STROBE); // Set illume
to: illume and strobe
XcBarcodeScanner.setFlashLightsMode(FlashMode.ILLUME_ONLY); // Set illume
to: illume only
```

Note: The illume light only turn on while scanning, the *STROBE* will output FLASH light which sync with image sensor shutter.

Config scan result text case switch

Use the following API to config case switch of scan result.

```
XcBarcodeScanner.setTextCase(String textCase);
```

All case options defined in the class TextCaseType.

```
public class TextCaseType {
    public static final String NONE = "NONE_CASE";
    public static final String UPPER = "UPPER_CASE";
    public static final String LOWER = "LOWER_CASE";
}
```

Sample code:

```
XcBarcodeScanner.setTextCase(TextCaseType.NONE); // No case switch.
XcBarcodeScanner.setTextCase(TextCaseType.UPPER); // Switch to Uppercase.
XcBarcodeScanner.setTextCase(TextCaseType.LOWER); // Switch to Lowercase.
```

Config prefix of barcode result

Use the following API to config prefix of barcode result.

```
XcBarcodeScanner.setTextPrefix(String prefix);
XcBarcodeScanner.setTextPrefix2(String prefix2);
```

Sample code:

```
XcBarcodeScanner.setTextPrefix("<"); // Config prefix as "<"
XcBarcodeScanner.setTextPrefix2(":"); // Config prefix2 as ":"
XcBarcodeScanner.setTextPrefix("Empty"); // Config prefix None.
```

Config suffix of barcode result

Use the following API to config suffix of barcode result.

```
XcBarcodeScanner.setTextSuffix(String prefix);
XcBarcodeScanner.setTextSuffix1(String prefix2);
```

Sample code:

```
XcBarcodeScanner.setTextSuffix(">"); // Config suffix as ">"
XcBarcodeScanner.setTextSuffix1(":"); // Config suffix2 as ":"
XcBarcodeScanner.setTextSuffix("Empty"); // Config suffix None.
```

Config interval of loopscan

Use the following API to config interval of loopscan.

```
setLoopScanInterval(int ms);
```

Sample code:

```
XcBarcodeScanner.setLoopScanInterval(100); // Config interval of loopscan as 100 ms.
```

Get running status of loopscan

Use the following API to get running status of loopscan.

```
boolean isLoopScanRunning();
```

Start/Stop loopscan

Use the following API to start or stop loopscan.

```
startLoopScan();
stopLoopScan();
```

Sample code:

```
// Start loopscan.
XcBarcodeScanner.setLoopScanInterval(100); // Set loopscan interval to 100 ms
XcBarcodeScanner.startLoopScan(); // start loopscan.

// Stop loopscan
if (XcBarcodeScanner.isLoopScanRunning()) { // Check if loopscan is running.
    XcBarcodeScanner.stopLoopScan(); // stop loopscan.
}
```

Config loopscan mode

Use the following API to config loopscan.

```
void enableLoopModeNotScan(boolean isEnabled);
```

Sample code:


```
// config loopscan.  
XcBarcodeScanner.enableLoopModeNotScan(true); // config loopscan.
```

Get status of multibarcodes

Use the following API to get the status of multibarcodes.

```
boolean isSupportMultiBarcodes();
```

Open/Close multibarcodes

Use the following API to open or close multibarcodes.

```
void enableMultiBarcodes(boolean isEnabled);
```

Sample code:

```
if(!XcBarcodeScanner.isSupportMultiBarcodes()){//Check if multibarcodes is  
close.  
    XcBarcodeScanner.enableMultiBarcodes(true); // open multibarcodes.  
}
```

Config multibarcodes

Use the following API to config multibarcodes options.

```
void setMultiBarcodes(int numberOfBarcodes, boolean fixedNumber);
```

The parameters:

```
numberOfBarcodes - Max barcodes in one shot. range: 1 ~ 20  
fixedNumber - true means fixed number of barcodes. false means non-fixed.
```

Note: Fixed number of barcodes means it will not output till the numberOfBarcodes barcodes been scanned. non-Fixed means it will output barcodes been scanned, it may smaller or equal to the numberOfBarcodes.

Sample code:

```
// Config numberOfBarcodes to 3, and be fixedNumber.  
XcBarcodeScanner.setMultiBarcodes(3, true);  
  
// Config numberOfBarcodes to 1  
XcBarcodeScanner.setMultiBarcodes(1, false);
```

Note: if the numberOfBarcodes been set to 1, it is single barcode mode, the fixedNumber option is meaningless.

Config region size of barcode scan

Use the following API to config the region size of barcode scanning. the typical usage is 1D barcode precise scanning.

```
void setScanRegionSize(int regionSize);
```

All supported region size defined in class RegionSizeType:

```
public class RegionSizeType {  
    public static final int VIEWSIZE_100 = 0; // 100% frame region.  
    public static final int VIEWSIZE_75 = 1; // 75% frame region.  
    public static final int VIEWSIZE_50 = 2; // 50% frame region.  
    public static final int VIEWSIZE_25 = 3; // 25% frame region.  
    public static final int VIEWSIZE_12 = 4; // 12% frame region.  
    public static final int VIEWSIZE_6 = 5; // 6% frame region.  
    public static final int VIEWSIZE_3 = 6; // 3% frame region.  
    public static final int VIEWSIZE_1 = 7; // 1% frame region.  
    public static final int VIEWSIZE_1D = 8; // 1D barcode region.  
}
```

Sample code:

```
// Config scan region to 1D barcode region.  
XcBarcodeScanner.setScanRegionSize(RegionSizeType.VIEWSIZE_1D);  
  
// Config barcode scan region to max region. (All 100% of framne)  
XcBarcodeScanner.setScanRegionSize(RegionSizeType.VIEWSIZE_100);
```

Note: if enabled multibarcodes and numberOfBarcodes more than 1, it is recommend to use 100% region of frame.

Get the last one decode image

Use the following API to get the last image to decode.

```
XCImage getLastDecodeImage();
```

The XCImage is the image class object retured.

Sample code:

```
import com.tools.XCImage;

public void getLastImage() {
    XCImage lastImg=XcBarcodeScanner.getLastDecodeImage();

    if(lastImg!=null){
        String infoStr="Width: "+lastImg.getWidth()+", Height: "+lastImg.getHeight()+", Stride: "+lastImg.getStride()+", size: "+lastImg.getData().length+" Bytes";
        showAlertDialog("Image Info:",infoStr,false,"OK",null);
    }else{
        showAlertDialog("Image Info:","No image!",false,"OK",null);
    }
}
```

Set custom BroadcastReceiver

Use the following API to configure the Action and Key for custom broadcasts. After successful configuration, the scan result can be received through the broadcast.

```
void setScanResultBroadcast(String action, String resultKey);
```

Sample code:

```
XcBarcodeScanner.setScanResultBroadcast("xxx.Action", "scanResultKey");
```

Disable/Enable Scan button

Due to differences in device design, the currently supported scanning buttons are: left side scanning button/right side scanning button/front scanning button/handheld handle scanning button. After disabling the scanning button function, pressing the scanning button will prevent scanning; After enabling the scanning button function, the button will restore the scanning function.

```
void setFrontScanKeyEnable(boolean isEnabled); // front scanning button

void setLeftScanKeyEnable(boolean isEnabled); // left side scanning button

void setRightScanKeyEnable(boolean isEnabled); // right side scanning button

void setPoGoScanKeyEnable(boolean isEnabled); // handheld handle scanning button
```

Sample code:

```
XcBarcodeScanner.setFrontScanKeyEnable(false); // Disable front scanning button

XcBarcodeScanner.setFrontScanKeyEnable(true); // Enable front scanning button
```

Config Custom scan key

Use the following API to config custom scan key.

```
boolean setCustomScanKey(int keyCode);
```

Sample code:

```
XcBarcodeScanner.setCustomScanKey(32); //config KEYCODE_D as scan key
```

Note: KEYCODE_DEL、KEYCODE_POWER、KEYCODE_CLEAR、KEYCODE_BACK、KEYCODE_HOME、KEYCODE_GRAVE cannot be set as the scan key.

Config Settings

Use the following API to import、export or reset settings.

```
void loadSettings();//import settings  
void saveSettings();//export settings  
void resetSettings();//reset settings
```

Config DataMatrix

Use the following API to config DataMatrix settings.

```
void setDataMatrixWithSeparators(boolean withSeparators);// Set whether to  
display separators  
void setDataMatrixMaxOutputLength(int maxLength);//set Max out length (0:no  
limit)
```

Sample code:

```
XcBarcodeScanner.setDataMatrixWithSeparators(true); //set to display  
separators  
  
XcBarcodeScanner.setDataMatrixMaxOutputLength(2); // set the max out lenght  
to 2
```

Config Fail notification

Use the following API to config fail notification

```
void setFailedNotification(String notification);
```

All supported notification value :

```
"Mute"      -----mute
"Sound"     -----sound
"Vib"       -----vib
"Sound/Vib" -----Sound/Vib
```

Sample code:

```
// Set up a sound prompt for failed scanning attempts.
XcBarcodeScanner.setFailedNotification("Sound");
```

Get the status of scan

Use the following API to get the status of scan .

```
void stopDecodeByMode(String scanMode);
```

Disable/Enable scan

Use the following API to disable or enable the scan function .

```
void enableScan(boolean isEnabled);
```

Sample code:

```
if(XcBarcodeScanner.isScanEnable()){//Check if scan is enable.
    XcBarcodeScanner.enableScan(false); //disable scan
}
```